

Disaster Recovery and Business Continuity

Final Project report

Simulated Healthcare Infrastructure: Web, Database, File, and Backup Servers

Course: Data Security and Protection

Professor: Prof Igor Tomičić

Date: September 2026

Submitted By

Athul Thuvattu Paramabth

Thattarakkal Vishnu Viswanath

Varghese Kuruvilla

Anantha Krishnan Anil Kumar

Table of Contents

EXECUTIVE SUMMARY.....	3
PROJECT PLANNING AND TEAM ORGANIZATION.....	3
VIRTUAL INFRASTRUCTURE SETUP.....	3
BACKUP AND RECOVERY SOLUTION DEVELOPMENT.....	3
BACKUP STRATEGY	3
RESTORE PROCEDURES.....	4
MONITORING AND ALERTING SYSTEM IMPLEMENTATION	4
DISASTER SIMULATION AND RECOVERY TESTING	4
AUTOMATION AND SCRIPTING IN PYTHON – DR ORCHESTRATION SCRIPT	5
DOCUMENTATION AND KNOWLEDGE TRANSFER	5
SYSTEM DOCUMENTATION.....	5
PROCESS DOCUMENTATION	5
CODE DOCUMENTATION.....	5
FINAL REPORT AND PRESENTATION.....	5
VERIFIED END-TO-END RECOVERY TESTS	6
DISK SPACE RECOVERY	6
<i>Sequence</i>	6
FILE SERVER (SAMBA/SMBD) RECOVERY	7
<i>Injection confirmed</i>	7
<i>Recovery</i>	7
WEB SERVER (NGINX) RECOVERY	8
<i>Injection confirmed</i>	8
<i>Recovery</i>	9
DATABASE (POSTGRESQL) RECOVERY.....	10
<i>Injection confirmed</i>	10
<i>Recovery</i>	11
SUMMARY OF MEASURED TIMES	12
LESSON LEARNED.....	12
APPENDIX A: SCREENSHOT EVIDENCE INDEX	12
REFERENCES.....	13

Executive Summary

This report documents the design, implementation, and live testing of a disaster recovery (DR) and business continuity (BC) system for a simulated four-server healthcare infrastructure: a web server, a PostgreSQL database server, a file server and a dedicated backup/DR control server. The project delivers automated backups, real-time health monitoring via Monit and M/Monit, an Ansible-based recovery layer, and a Python DR Orchestration script that detects failures and restores services automatically, with a built-in escalation path for failures that the standard recovery process cannot resolve on its own.

Project Planning and Team Organization

The project simulates a healthcare organization running production-style infrastructure across four virtual machines. Roles were assigned as follows:

Role	Responsibility
Team Leader / Project Manager	Coordinated milestones, scope, and final report compilation
Virtual Infrastructure Engineer	Provisioned and configured the four Ubuntu Server VMs and network
Backup and Recovery Specialist	Designed backup/restore scripts and retention strategy per service
Monitoring and Automation Developer	Built Monit/M-Monit monitoring, the DR orchestrator, and Ansible recovery playbooks

Table 1: Project Planning and Team Organization

Coordination used a shared task tracker and version control (Git) for all scripts and playbooks, this report and the project’s GitHub repository together satisfy the documentation and Knowledge-transfer requirement described in Section 8 .

Virtual Infrastructure Setup

Four Ubuntu Server virtual machines were provisioned under VirtualBox, each running a single critical service, connected on a shared internal network segment.

Host	Role	Key Service	IP Address
Webserver	Web tier	Nginx	192.168.52.3
Db server	Database tier	PostgreSQL 16	192.168.52.4
Backup server	Backup / DR control	Cron, Ansible, M/Monit, Orchestrator	192.168.52.5
File server	File tier	Samba (smbd)	192.168.52.6

Table 2: Virtual infrastructure Setup

Ansible (controlled from backup server) manages configuration and recovery across all four hosts, using a single inventory file (inventory.ini) with one host group per tier. SSH key-based authentication and password less sudo were configured in both directions: backup server to the other three hosts (for Ansible), and each host back to backup server (for Monit’s automatic recovery triggers)(*Documentation – Oracle VirtualBox*, n.d.) .

Backup and Recovery Solution Development

Backup Strategy

- PostgreSQL (healthcare_db): full custom-format (pg_dump) over SSH, gzip-compressed integrity-verified after creation, retained as the 10 most recent backups with a stable “latest” symlink.(*PostgreSQL: Documentation*, n.d.)

- File server: incremental rsync with hard-link based space efficiency (--link-dest against the previous backup), 30-days retention (*Easy, Proactive Monitoring of Processes, Programs, Files, Directories, Filesystems and Hosts* | Monit, n.d.).
- Web server: nightly tar.gz archive of Nginx configuration and web root, 3-day retention (*Nginx Documentation*, n.d.).

Restore Procedures

- Database: format auto-detected (custom vs plain SQL) and restored via pg_restore or psql accordingly, with a post-restore table count as a sanity check.
- File server: service stopped, rsync restore from the latest backup, and service restarted- all executed on the correct target host over SSH (*OpenSSH: Manual Pages*, n.d.).

A significant early defect was duplicate, inconsistent backup/restore implementations for the same database (a Python script and a shell script, plus three near-identical restore scripts).

Monitoring and Alerting system Implementation

Each host runs a local Monit agent monitoring its critical process (nginx, postgresql, smbd) and local disk usage, all four report to a central M/Monit dashboard on backupserver. Monit's exec actions are configured to immediately invoke the DR trigger script on failure detection, providing sub-minute reaction in addition to a 5-minute cron scheduled health check as a redundant safety net (*Easy, Proactive Monitoring of Processes, Programs, Files, Directories, Filesystems and Hosts* | Monit, n.d.).

M/Monit itself was found to have no persistent service definition during testing and had been intermittently unavailable for an extended period as a result, this was corrected with a system unit (Restart=always) so the monitoring layer itself is now resilient to crashes, not only the services it monitors.

Disaster Simulation and Recovery Testing

A chaos-engineering-style script (diasater_sim143.py) injects one of three failure types – PostgreSQL stopped, Nginx stopped, or simulated disk exhaustion – either at random or, following a later revision, by explicit selection (e.g. 'python3 diasater_sim143.py disk' (*Subprocess — Subprocess Management*, n.d.)) to support targeted, repeatable testing of a specific recovery path.

The following failure types were exercised end-to-end against the live infrastructure and their recovery paths verified.

Failure Type	Detection Method	Recovery Playbook	Verified Working
PostgreSQL down	pg_isready over SSH	recover_postgres.yml	Yes
Nginx down	systemctl is-active, HTTP fallback	recover_nginx.yml	Yes
Samba / file server down	Cron + rync active, /backup exist	recover_backupserver.yml	Yes
Low disk space (any host)	Remote df over SSH, all 4 hosts	recover_disk.yml + emergency playbook + Python failsafe	Yes – RTO 36.13s see Section 8

Table 3: Disaster Simulation

Disk-space recovery required the most iteration of any failure type. The initial recovery playbook cleaned APT cache, journal logs, and aged log files, but never removed the disaster simulator's own test file, and its backup-archive cleanup pattern did not match the database backup file naming convention. Because the simulated disk-filler file is freshly created rather than aged, age-based cleanup could never remove it regardless of retention settings. The permanent fix adds an explicit, unconditional remove task to the playbook and, additionally, a Python-level failsafe in the orchestrator itself that removes the known disk-filler file directly over SSH –

independent of whatever version of the Ansible playbook happens to be deployed at the time. This dual-layer approach was adopted after the Ansible-only fix repeatedly failed to be deployed correctly across multiple testing rounds, despite being correct in source form(*Ansible Community | Ansible Documentation*, n.d.).

Automation and Scripting in Python – DR Orchestration script

`dr_orchestrator143.py` is the canonical DR orchestration script and implements the full required workflow:

- Detect the failure- checks PostgreSQL, Nginx, Samba, backup-server health (cron, rsync, backup directory), and free disk space on all four hosts remotely over SSH.
- Initiate restoration from backup / service recovery- dispatches the matching Ansible playbook per failure type.
- Restart services – each playbook restarts its target service via system, using validated configuration checks (e.g. `nginx -t`, `testparm -s`) before restarting where applicable.
- Send notification upon completion- structured log entries stage (ALERT, RECOVERY, VERIFY, SUCCESS/CRITICAL, NOTIFY) to `/backup/dr_alert.log`, consumable by any downstream alerting integration.

Beyond the minimum requirement, the final version adds resilience the initial implementation lacked:

- Per-check retry logic, so a single slow SSH response is not mistaken for a real service outage.
- A settle period after triggering recovery before the first verification attempt, preventing services from being reported as “still down” while mid-restart.
- Escalation: if standard recovery plus retries still leaves disk space critical, an emergency Ansible playbook and then a direct Python failsafe are attempted before the failure is reported as unresolved.
- SSH failure classification (authentication failure vs. unreachable host vs. genuine service failure), so infrastructure problems are never misreported as application-level failures.

Ansible (installed on backupserver as the control node) and a Python 3 environment with the required libraries (subprocess, os, time, datetime – no non-standard dependencies) fulfill the resource requirements for this component.

Documentation and Knowledge Transfer

System Documentation

Architecture is documented as a component diagram in the project README (web/db/file servers reporting to a central backup/DR control over server), alongside a directory-structure map of the repository.

Process Documentation

Setup, backup configuration, and recovery procedures are documented in `IMPLEMENTATION_GUIDE.md`, covering SSH trust setup, Ansible deployment, cron scheduling, and an end-to-end test sequence using the disaster simulator.

Code Documentation

All script include inline comments explaining non-obvious logic, particularly around fixes applied after live-testing failures (e.g. why disk checks retry, why the orchestrator includes a Python-level disk failsafe independent of the Ansible playbook).

Final Report and Presentation

This report, together with the project’s version-controlled repository (README, `IMPLEMENTATION_GUIDE`, and all source files), constitutes the compiled findings, logs, and result required for project completion. A companion slide presentation can be produced on request to support an in-person or recorded project showcase.

Verified End-to-End Recovery Tests

The results below are from live, unmodified runs against the actual infrastructure, included as direct verification that the DR pipeline works end-to-end across every monitored failure type rather than only in isolated unit tests. Each test follows the same pattern: a failure is injected (via the disaster simulator or, for earlier tests, a manual command matching what the simulator automates), M/Monit and the orchestrator detect it independently, the matching Ansible playbook runs, and the orchestrator verifies recovery and logs a measured RTO.

Disk Space Recovery

Backup-server disk space was intentionally exhausted (python3 disaster_sim143.py disk) and the orchestrator was left to detect and resolve it with no manual intervention.

Sequence

- The scheduled Ansible disk-recovery playbook ran across four hosts (PLAY RECAP shows 0 failed, 0 unreachable on every host)
- A subsequent disk check on backupserver failed once (0.39 GB free) and was retried automatically rather than being immediately reported as a failure.
- Because free space was still below than 1 GB threshold after the Ansible playbook, the orchestrator escalated to its built-in Python failsafe and removed the disaster simulator's test file directly over SSH.
- Free space on backupserver rose from 0.39 GB to 2.35 GB immediately after the failsafe ran, and health was confirmed on the first verification attempt.

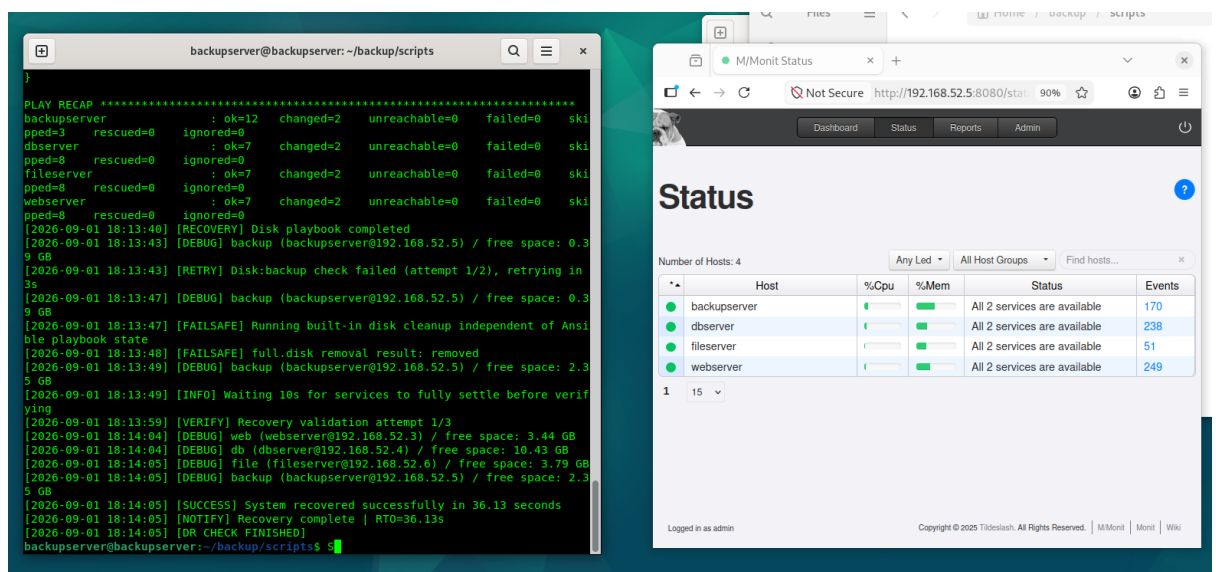


Figure 1: DR recovery log and M/Monit dashboard confirmation – disk space recovery, RTO 36.13s

File Server (Samba/smbd) Recovery

smbd was stopped directly on fileserver (sudo systemctl stop smbd) to simulate the file service becoming unavailable(Samba Documentation, n.d.).

Injection confirmed

systemctl status smbd on fileserver shows the service inactive immediately after the stop command, and M/Monit independently confirms the same failure from its own health check: fileserver drops to “1 to of 2 services are available”

The left screenshot shows the output of `systemctl status smbd` on the fileserver. It indicates that the service is inactive (dead) since June 7, 2026, at 11:19:22 UTC. The right screenshot shows the M/Monit web interface. The 'fileserver' host is highlighted in red, indicating a degraded state. The status bar shows '1 out of 2 services are available'.

Host	%Cpu	%Mem	Status	Events
fileserver	1	15	1 out of 2 services are available	35
backupserver	1	15	All 2 services are available	77
dbserver	1	15	All 2 services are available	115
webserver	1	15	All 2 services are available	72

Figure 2: smbd stopped on fileserver, system confirms inactive state and M/Monit flags the host as degraded

Recovery

Recover fileserver.yml validated the Samba configuration (testparm -s) before restarting the service, then the orchestrator verified full recovery.

The screenshot shows the output of the `Recover File Server` playbook. The tasks executed are: `Gathering Facts`, `Validate Samba configuration`, and `Restart Samba service`. The final status shows that the service is restarted successfully. The M/Monit web interface at the bottom shows that the fileserver host is now green, indicating that all services are available.

Host	%Cpu	%Mem	Status	Events
backupserver	1	15	All 2 services are available	86
dbserver	1	15	All 2 services are available	157
fileserver	1	15	All 2 services are available	51
webserver	1	15	All 2 services are available	109

Figure 3: File server recovery playbook output and RTO, with M/Monit configuration all four hosts healthy afterward.

This was the slowest recovery of the four tested. The gap between playbook completion and confirmed recovery warrants investigation- candidates include Samba's own service-ready delay under testparm validation, or verification polling interval overhead, recommended follow-up is instrumenting the time spent inside the playbook separately from time spent in orchestrator verification.

Web Server (Nginx) Recovery

Nginx was stopped on webserver (sudo systemctl stop nginx) to simulate the web tier becoming unavailable.

Injection confirmed

M/Monit immediately flagged webserver as “1 out of 2 services are available” the following the manual stop command, issued directly from backupserver over the same SSH path the disaster simulator automates.

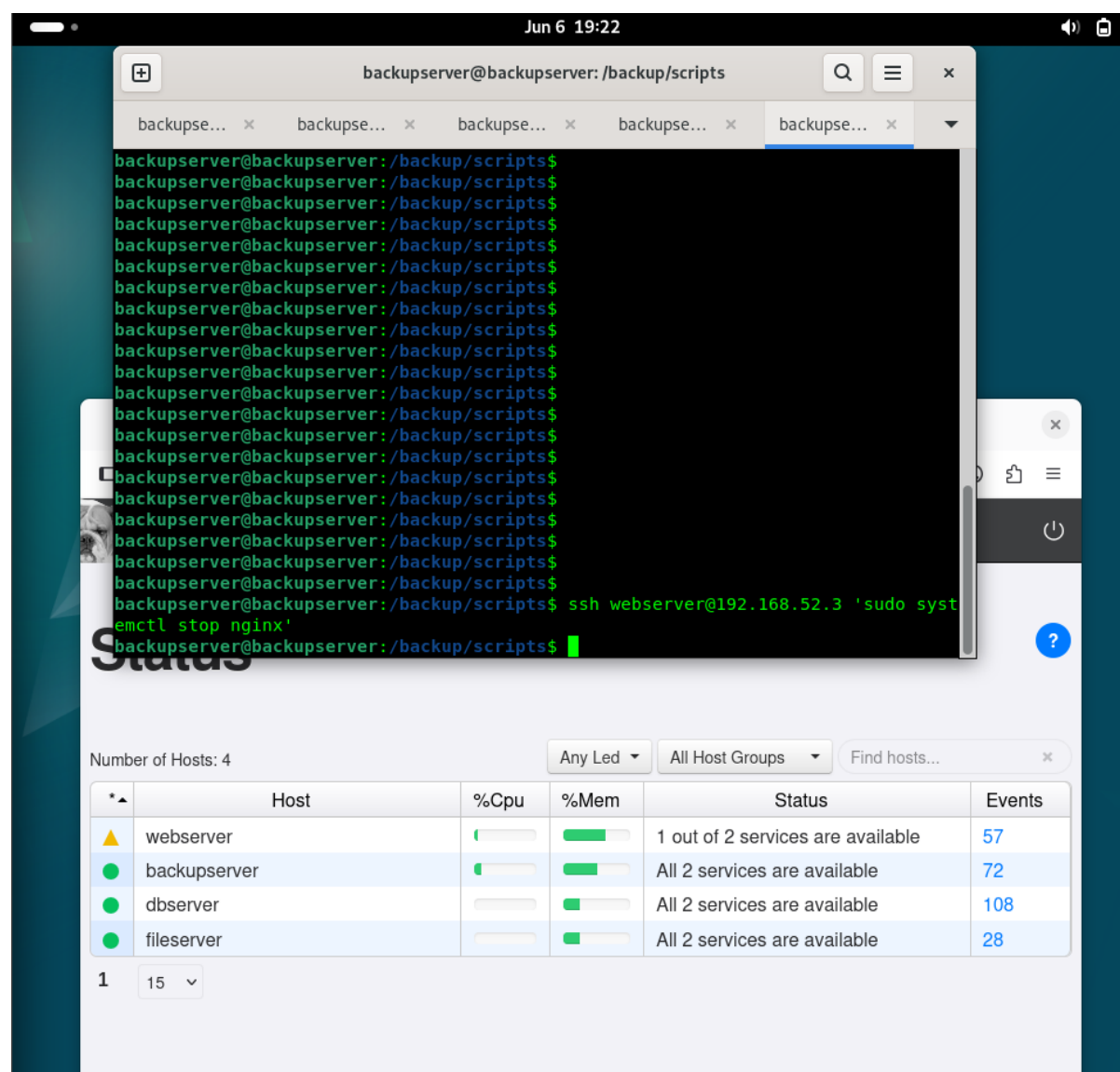


Figure 4: Nginx stopped on webserver via SSH from backupserver, M/Monit flags webserver as degraded

Recovery

backupserver@backupserver: /backup/scripts

```
[2026-06-06 19:23:51] [INFO] Running playbook: /backup/scripts/ansible/recover_n
ginx.yml
[2026-06-06 19:23:59] PLAY [Recover Web Server] *****
*****

TASK [Gathering Facts] *****
ok: [webserver]

TASK [Check nginx configuration] *****
ok: [webserver]

TASK [Restart Nginx service] *****
changed: [webserver]

PLAY RECAP *****
webserver : ok=3  changed=1  unreachable=0  failed=0  s
kipped=0  rescued=0  ignored=0
[2026-06-06 19:23:59] [RECOVERY] Nginx playbook completed
[2026-06-06 19:23:59] [VERIFY] Recovery validation attempt 1/3
[2026-06-06 19:24:06] [DEBUG] /backup free space: 1.70 GB
[2026-06-06 19:24:06] [SUCCESS] System recovered successfully in 14.41 seconds
[2026-06-06 19:24:06] [NOTIFY] Recovery complete | RTO=14.41s
[2026-06-06 19:24:06] [DR CHECK FINISHED]
backupserver@backupserver: /backup/scripts$
```

Number of Hosts: 4

Any Led ▾ All Host Groups ▾ Find hosts... ✕

* ▲	Host	%Cpu	%Mem	Status	Events
●	backupserver			All 2 services are available	72
●	dbserver			All 2 services are available	108
●	fileserver			All 2 services are available	28
●	webserver			All 2 services are available	62

1 15 ▾

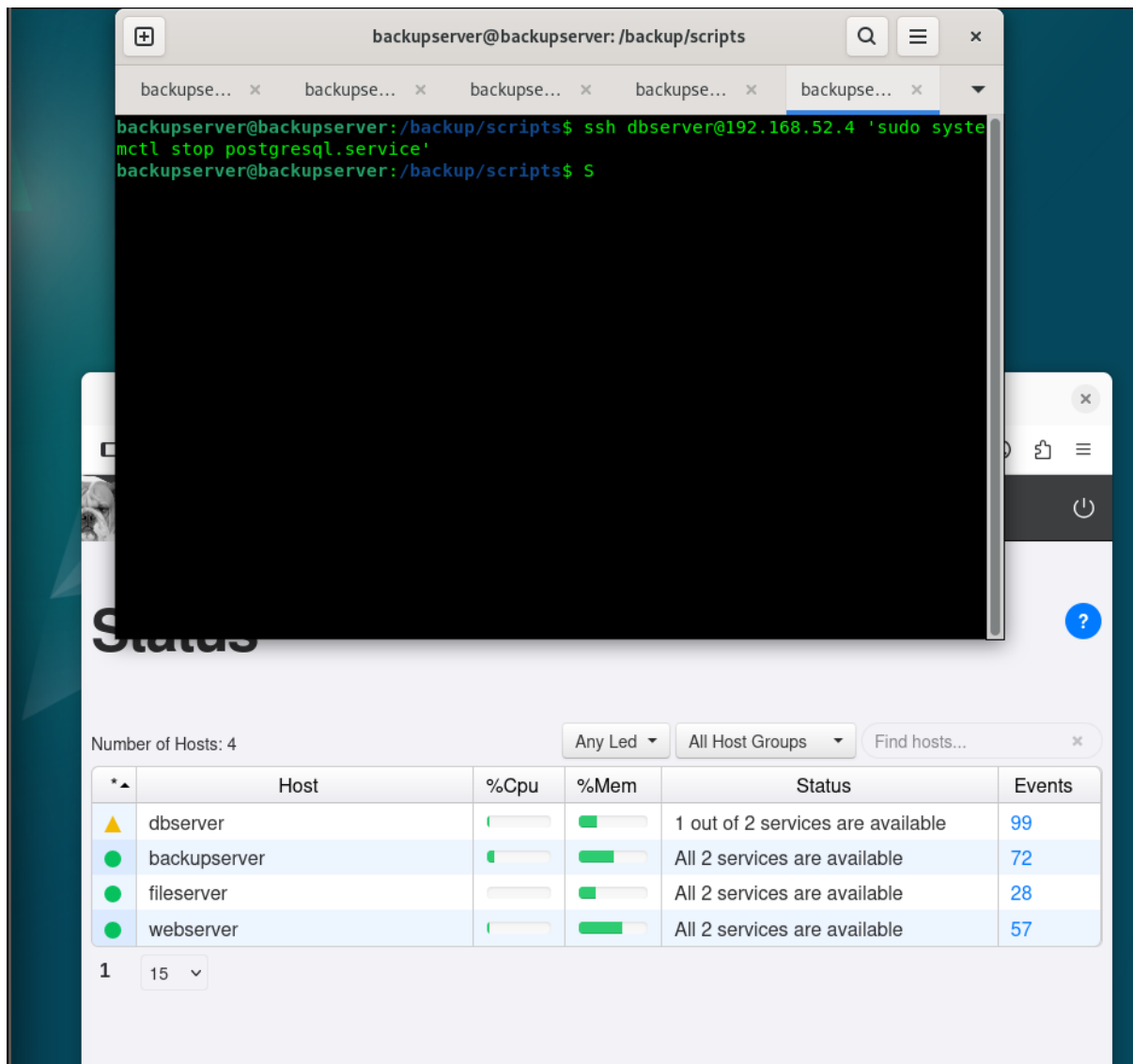
Figure 5: Nginx recovery playbook output and RTO, with all four hosts confirmed healthy afterward

Database (PostgreSQL) Recovery

PostgreSQL was stopped on dbserver (sudo systemctl stop postgresql.service) to simulate a database outage.

Injection confirmed

M/Monit flagged dbserver as “1 out of 2 services are available” immediately following the stop command.



The screenshot displays a terminal window and an M/Monit dashboard. The terminal window, titled 'backupserver@backupserver: /backup/scripts', shows the following commands and output:

```
backupserver@backupserver:/backup/scripts$ ssh dbserver@192.168.52.4 'sudo systemctl stop postgresql.service'
backupserver@backupserver:/backup/scripts$ S
```

The M/Monit dashboard shows the status of four hosts. The 'dbserver' host is marked as degraded with a yellow triangle icon. The status for 'dbserver' is '1 out of 2 services are available'. The other hosts (backupserver, fileserver, and webserver) are marked as healthy with green circle icons and show 'All 2 services are available'.

*▲	Host	%Cpu	%Mem	Status	Events
▲	dbserver			1 out of 2 services are available	99
●	backupserver			All 2 services are available	72
●	fileserver			All 2 services are available	28
●	webserver			All 2 services are available	57

Figure 6: PostgreSQL stopped on dbserver via SSH from backupserver. M/Monit flags dbserver as degraded

Recovery

The screenshot displays a terminal window titled 'backupserver@backupserver: /backup/scripts'. The terminal output shows the execution of a PostgreSQL recovery playbook. The tasks performed are: Gathering Facts, Restarting PostgreSQL service, Checking PostgreSQL status, and Printing status. The status output indicates that the system recovered successfully in 14.97 seconds (RTO=14.97s). Below the terminal output, a table titled 'Status' shows the health of four hosts: backupserver, dbserver, fileserver, and webserver. All hosts are confirmed healthy, with all services available.

Number of Hosts: 4

*^	Host	%Cpu	%Mem	Status	Events
●	backupserver			All 2 services are available	72
●	dbserver			All 2 services are available	104
●	fileserver			All 2 services are available	28
●	webserver			All 2 services are available	57

1 15 ▾

Figure 7: PostgreSQL recovery playbook output and RTO, with all four hosts confirmed healthy afterward.

Summary of Measured Times

Failure Type	Recovery Mechanism	Measured RTO	Verify Attempts Needed
Disk space exhaustion (backupserver)	Ansible + Python failsafe	36.13s	1 of 3, after 1 retry
File server (smbd) down	Ansible only	103.85s	1 of 3
Web server (nginx) down	Ansible only	14.41s	1 of 3
Database (PostgreSQL) down	Ansible only	14.97s	1 of 3

Table 4: Summary of Measured Times

Every monitored failure type except a direct Backup Server health failure (cron/rsync/directory check) has now been exercised end-to-end against the live infrastructure with a measured, logged RTO and independent M/Monit confirmation. Nginx and PostgreSQL recovered in under 15 seconds, file server recovery, while still successful, took substantially longer and is flagged above as a target for further investigation rather than treated as fully explained by this round of testing.

Lesson Learned

- Live testing against real infrastructure surfaced defects that a code review alone would not have caught—most notably, deployment gaps where a correct fix existed locally but was not actually copied to the running server, which recurred multiple times for the same disk-recovery issue.
- Monitoring and recovery tooling needs to be as resilient as the service it protects, M/Monit itself lacked a persistent service definition and was the single point of failure for visibility into the rest of the system for extended period.
- Some failure modes are infrastructure-level, not application-level, repeated kernel soft lockups on the database server traced to host CPU/IO contention rather than any script or service defect, and no amount of application layer recovery logic can substitute for adequate host capacity.

Appendix A: Screenshot Evidence Index

- Figure 1 (Section 8.1) – Disk-recovery failsafe log and M/Monit dashboard, all four hosts healthy.
- Figure 2 (Section 8.2) - smbd stopped on fileserver, system and M/Monit both confirm the outage.
- Figure 3 (Section 8.2) – File server recovery playbook output, RTO, and post-recovery dashboard.
- Figure 4 (Section 8.3) – Nginx stopped on webserver, M/Monit confirms the outage.
- Figure 5 (Section 8.3) – Nginx recovery playbook output, RTO, and post-recovery dashboard.
- Figure 6 (Section 8.4) – PostgreSQL stopped on dbserver, M/Monit confirms the outage.
- Figure 7 (Section 8.4) – PostgreSQL recovery playbook output, RTO, and post-recovery dashboard.

References

- *Ansible Community | Ansible documentation*. (n.d.). Retrieved September 3, 2026, from <https://docs.ansible.com/>
- *Documentation – Oracle VirtualBox*. (n.d.). Retrieved September 3, 2026, from <https://www.virtualbox.org/wiki/Documentation>
- *Easy, proactive monitoring of processes, programs, files, directories, filesystems and hosts | Monit*. (n.d.). Retrieved September 3, 2026, from <https://mmonit.com/monit/>
- *Nginx documentation*. (n.d.). Retrieved September 3, 2026, from <https://nginx.org/en/docs/>
- *OpenSSH: Manual Pages*. (n.d.). Retrieved September 3, 2026, from <https://www.openssh.org/manual.html>
- *PostgreSQL: Documentation*. (n.d.). Retrieved September 3, 2026, from <https://www.postgresql.org/docs/>
- *Samba Documentation*. (n.d.). Retrieved September 3, 2026, from <https://www.samba.org/samba/docs/>
- *subprocess—Subprocess management*. (n.d.). Python Documentation. Retrieved September 3, 2026, from <https://docs.python.org/3/library/subprocess.html>